

Report on Assignment 3

3D Object Loading, Rendering, and Animation in OpenGL

Course name : Fundamental of Computer Graphics
Course Code : DVA338
Examination : PMO1

Youssef EL OTMANI : < youssefelotmanietudesup@gmail.com >

Abstract

This project includes a 3D rendering system in OpenGL, where it mainly focuses on loading objects, using instancing and generating animations. With the help of the provided obj library, models are loaded from obj files and their materials and textures which takes the place of previously defined meshes. Shaders in the system handle the diffuse, ambient and specular parts of lighting by mixing color and textures. We used object instancing to display thousands of models and every copy has its own changes and frame-to-frame animations. We are able to construct a dynamic scene featuring repeating terrain tiles, light sources, and camera controls for both first- and third-person views.

Tables of Content

I) Project Presentation **3**
II) Program Design **3**
Data Structures **4**
Shader Management and Binding 6
Transformation and Animation 6
III) Implementation **7**
Core Functions **7**
Optimizations and Creative Approaches 9
IV) Testing and Results **10**
V) Conclusion and Reflections **15**
References **15**

I) Project Presentation

This assignment aimed to develop a fast and handy 3D rendering program with OpenGL that is able to load objects, support instancing, animation and scene building. In particular, the task

involved changing the custom *Mesh* type with the *obj* type given in the added library to allow rendering objects with materials and textures. Efficiency in managing objects, using materials correctly and connecting with GLSL shaders was the main focus of the project. Besides this, instancing and animation allowed us to generate impressive large scenes full of moving things that came together seamlessly. Some of the goals were :

- Loading *.obj* models using obj_create and obj_proc.
- Using textures and material properties to improve objects' appearance in shaders.
- Creating an animated environment that can be looked at from both first and third-person perspectives.
- Creating multiple object *instances* with their own transforms and animations.

While trying to implement these goals, i encountered many challenges like :

- Efficient rendering of multiple objects without performance drops.
- Managing animation and looping with no floating-point drift or state accumulation.
- Real-time switching between projection types and camera views.

II) Program Design

The program uses OpenGL to render in real time and displays a 3D image after the obj library has been used to load the model with lighting and material support. It employs OpenGL and GLSL to manage item shape changes, their material settings and the dynamics of lighting. The system consists of several important elements which are:

- Model Loader: It uses the obj_create() and obj_proc() functions to link .obj models, .mtl and .tga textures together in a structure called struct obj.
- Rendering System: Shows all models that have been loaded in **sceneObjects** by giving their material and geometry information to the GPU.
- Camera and Projection: Computes ViewMatrix and PerspectiveProjectionMatrix using a Camera struct, allowing flexible viewpoint control.
- Shader Management: Compiles and links multiple shaders and binds their input attributes and uniform locations to the object properties.

Data Structures

❖ struct obj (from obj.h)

This structure contains the full data needed for a 3D model:

- Geometry: Vertices, normals, tangents, texture coordinates.
- Materials: including diffuse, ambient, specular, and texture information.
- OpenGL Handles: VAO, VBO, IBOs per surface for efficient rendering.
- Shader Bindings: Locations for vertex attributes and uniforms (position, normal, tangent, etc.).

The object is rendered using obj_render(obj*), which internally uses the VAO and bound material properties to draw all associated surfaces.

❖ std::vector<obj*> sceneObjects

This is the user-defined container in main.cpp holding all 3D objects to render in the scene. Objects are loaded once and reused across frames: (here X represent any model)

```
obj* X = obj_create("../X.obj");
if (!X) {
      fprintf(stderr, "Error");
      exit(EXIT_FAILURE);
}
else
      printf("X loaded.\n");

obj_proc(X);

sceneObjects.push_back(X);
```

❖ struct Instance

In main.cpp, the definition of this struct applies to each time an object is rendered.

```
struct Instance {
    obj* model;                    // Shared geometry
    glm::vec3 position;
    glm::vec3 rotation;
    glm::vec3 scale;
    glm::mat4 modelMatrix;
    GLuint shaderProgram;

    glm::vec3 translationPerFrame;
    glm::vec3 rotationPerFrame;
    glm::vec3 scalePerFrame;
    int loopDuration;
};
```

All instances have geometry from obj*, yet they each have their own transformations and animations.

Instances are stored in `std::vector<Instance> Instances;`

- **struct light**

Represents a basic point light:

```
struct light {
    glm::vec3 position;
    glm::vec3 ambient;
    glm::vec3 diffuse;
    glm::vec3 specular;
};
```

All Lights struct are stored in `std::vector<light*> lightsFolder`.

- **struct Spotlight**

Put in the spotlight position, direction and defines how much it is dimmed down and the animation.

```
struct Spotlight {
    glm::vec3 position, direction;
    float cutOff, outerCutOff;
    glm::vec3 ambient, diffuse, specular;
    glm::mat4 modelMatrix;
    glm::vec3 transAnim, rotAnim, scaleAnim;
};
```

All Lights struct are stored in `std::vector<Spotlight*> spotlightsFolder.`

- **struct Camera (from camera.cpp)**

Used to calculate the view and projection matrices.

```
struct Camera {
    Vector position;
    Vector rotation;
    float fov, nearPlane, farPlane;
};
```

The camera is set up each frame using ViewMatrix() and PerspectiveProjectionMatrix() with the use of this variable.

Shader Management and Binding

Shaders are made using prepareShaderProgram() and stored in an array.

```
GLuint shaderPrograms[3];
```

Every shader is bound to an obj using obj_set_vert_loc() to bind vertex attributes and obj_set_prop_loc() to bind material properties.

The bindingShaderVariables() function is called in every frame and it sets uniforms for lighting (multiple lights & spotlights), view position, Material color or texture maps.

Transformation and Animation

Transform matrices are constructed using :

```
glm::mat4 cal(position, rotation.x, rotation.y, rotation.z, scale);
```

and it's defined as follows :

```
glm::mat4 cal(glm::vec3 pos, float x, float y, float z, glm::vec3 scales) {
return glm::translate(glm::mat4(1.0f), pos)
      * (glm::rotate(glm::mat4(1.0f), glm::radians((z)), glm::vec3(0, 0, 1))
      * (glm::rotate(glm::mat4(1.0f), glm::radians((y)), glm::vec3(0, 1, 0))
      * (glm::rotate(glm::mat4(1.0f), glm::radians((x)), glm::vec3(1, 0, 0))
      * glm::scale(glm::mat4(1.0f), scales))));
}
```

Animations are frame-based :

```
glm::vec3 pos = instance.position + ratio * instance.translationPerFrame;
glm::vec3 rot = instance.rotation + ratio * instance.rotationPerFrame;
glm::vec3 scl = instance.scale + ratio * instance.scalePerFrame;
```

`getAnimatedTransform()` implements the final model matrix for the rendering.

At each frame, a display() function is called, and it :

- Make sure both the screen and the depth buffer are cleared.
- The camera's view and projection matrices are generated
- Go through every instance in the Instances class and Calculate animated transform, Bind shader variables and Call obj_render().
- Light sources are passed as uniforms per frame.

III) Implementation

Core Functions

❖ Instances, light and spotlight creation

Instances of objects are not duplicated but we rather use the same obj* pointer for multiple different Instances of the same model, which are created using the custom addInstance() function:

```
static void addInstance(obj* model, glm::vec3 position, glm::vec3
rotation, glm::vec3 scale, int shader);
```

Each call creates an Instance, assigns it a transformation, and stores its own model matrix and pushes it into *Instances*.

For example, the road tiles and lamps:

```
for (int i = 0; i < 10; i++) {
addInstance(sceneObjects[3], { -2, -2, (float) -10 - i * 24 }, { 0, 0, 0 }, { 1,
1 ,1 }, 1); // road
addInstance(sceneObjects[2], { -5.85, 0,(float)0 - i * 24 }, { 0, 180, 0 }, { 1,
1 ,1 }, 1);              // left lamp
addInstance(sceneObjects[2], { 1.85, 0,(float)-5 - i * 24 }, { 0, 0, 0 }, { 1, 1
,1 }, 1);                     // right lamp
}
```

Similarly, Point lights are added using `addlight()`, which creates a light* object and pushes it into lightsFolder, and the same for spotlight with `addSpotlight()`, stored in `spotlightsFolder`.

```
void addlight(glm::vec3 position, glm::vec3 ambient, glm::vec3 diffuse,
glm::vec3 specular);
void addSpotlight(glm::vec3 pos, glm::vec3 dir, float cutoff, float
outer, glm::vec3 amb, glm::vec3 diff, glm::vec3 spec);
```

We make the spotlight support animated translation movement with :

```
void addAnimationSpotLight(Spotlight* spotLightTemp, glm::vec3
translation) {
```

```
	if (spotLightTemp == nullptr) {
		fprintf(stderr, "[ANIMATION] Error : Instance does not
exist.\n");
		exit(EXIT_FAILURE);
	}
	else {
		spotLightTemp->transAnim = translation;
	}
}
```

Spotlights are updated each frame using:

```
spotlight->modelMatrix = transformAnimLight(*spotlight);
```

This keeps them in sync with the moving scene (like the street lamps sliding toward the camera).

❖ Animation

```
glm::mat4 getAnimatedTransform(const Instance& inst) {
	float ratio = fmod(currentFrame, Duration) / Duration;

	glm::vec3 pos = inst.position + inst.translationPerFrame * ratio;
	glm::vec3 rot = inst.rotation + inst.rotationPerFrame * ratio;
	glm::vec3 scl = inst.scale + inst.scalePerFrame * ratio;

	return cal(pos, rot.x, rot.y, rot.z, scl);

}
```

❖ Viewpoint switching

We implement two cameras positions settings as follows and a Camera pointer to the active one (third person viewpoint perspective by default)

```
Camera cam_third_person = { {0,0,5}, {0,0,0}, 60, 1, 1000 };
Camera cam_first_person = { {0,0,0}, {0,0,-1}, 60, 1, 1000 };
Camera* activeCam = &cam_third_person;
```

and we are able to switch between them, with this, implemented in the keypress function :

```
case 'v':
	if (activeCam == &cam_third_person)
		activeCam = &cam_first_person;
	else
		activeCam = &cam_third_person;
	break;
```

The *ViewMatrix()* and *PerspectiveProjectionMatrix()* functions are called for every frame by using ***activeCam**, meaning the transformations and shader bindings automatically update based on the current view.

❖ Cadillac Headlight Toggle

In the scene, Cadillac's headlights are intended as the first spotlights seen in front of the car. Turning on the spotlight for each car depends on the global boolean called `carSpotlights`.

Inside the keypress function, we added the fact that pressing `'E'` toggles this variable,

```
case 'e': carSpotlights = !carSpotlights; break;
```

We then send it to the shader for each frame with :

```
glUniform1i(locUseCarSpotlights, (int)carSpotlights);
```

In the fragment shader, spotlights 0 and 1 are skipped when carSpotlights == 0:

```
if (!(useCarSpotlights == 0 && (i == 0 || i == 1))) { ... }
```

❖ Sending Phong Material to the Shader

For each obj, we link its materials to the shader using:

```
obj_set_prop_loc(obj, OBJ_KD, loc_KdColor, loc_KdTexture, -1);  // Diffuse
obj_set_prop_loc(obj, OBJ_KA, loc_KaColor, -1, -1);              // Ambient
obj_set_prop_loc(obj, OBJ_KS, loc_KsColor, -1, -1);              // Specular
```

These bind the uniform locations of each Phong term (color and texture) to the GPU shader.

Then in the shader (GLSL):

```
uniform vec3 matDiffuse;
uniform vec3 matAmbient;
uniform vec3 matSpecular;
uniform sampler2D NormalTexture;
```

The fragment shader samples textures if present or falls back to color values for each term.

Optimizations and Creative Approaches

- **Infinite Terrain Scrolling:** Instead of generating an endless road procedurally, a fixed number of road and lamp post objects are looped using frame-based translation and modulo logic, creating the illusion of an infinite world with minimal asset cost.
- **Frame-Based Animation**: To prevent cumulative floating-point drift, animations are computed from scratch using the frame index. This makes animations deterministic, loopable, and highly parallelizable.
- **Multiple Shaders:** Multiple shaders are combined and chosen at the time of the run by using keyboard shortcuts. With this, we can easily try out and test several different lighting models instantly (Phong, cartoon or NormalAsColor), wich helped pinpoint problems while coding (if a problem occurred no matter the shaders used, the problem was most of the time in the main, and if some shaders work and others don't then, the problem is from the shader itself or the uniforms needed and sent in the main for that shader)
- **Spotlight Simulation:** Spotlights simulate car headlights and street lamps. Direction, cone angles, and toggles are handled in shaders using uniform inputs and controlled via input keys, enriching realism.

IV) Testing and Results



third person view perspective



we can see that the light from the lamp shows on the cadillac



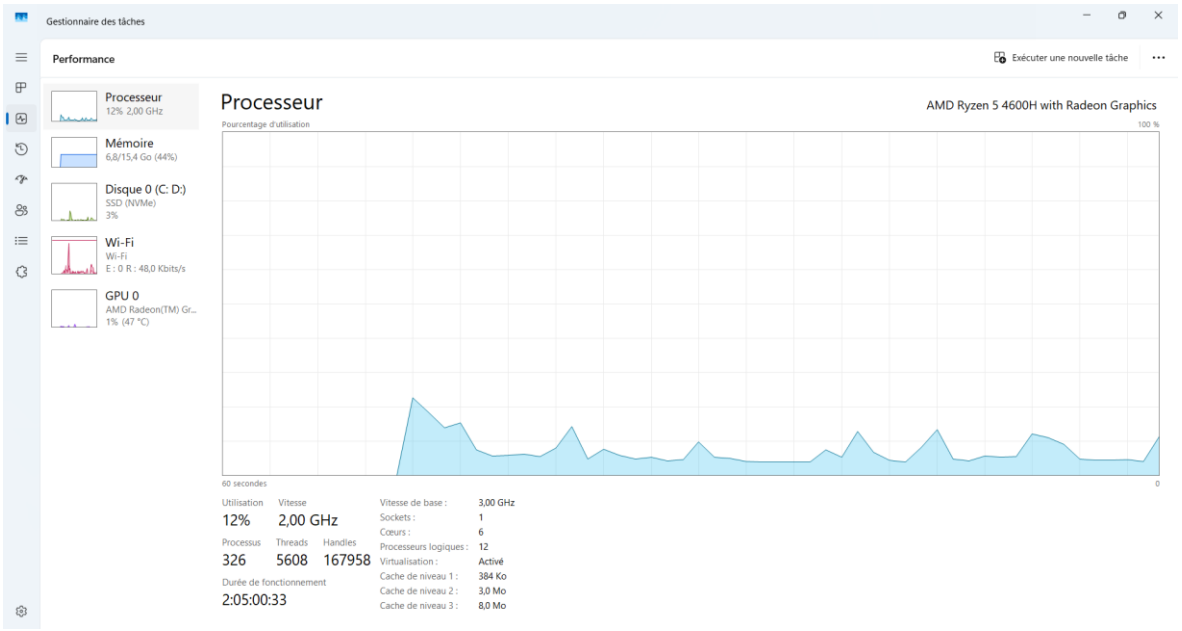
first person view perspective



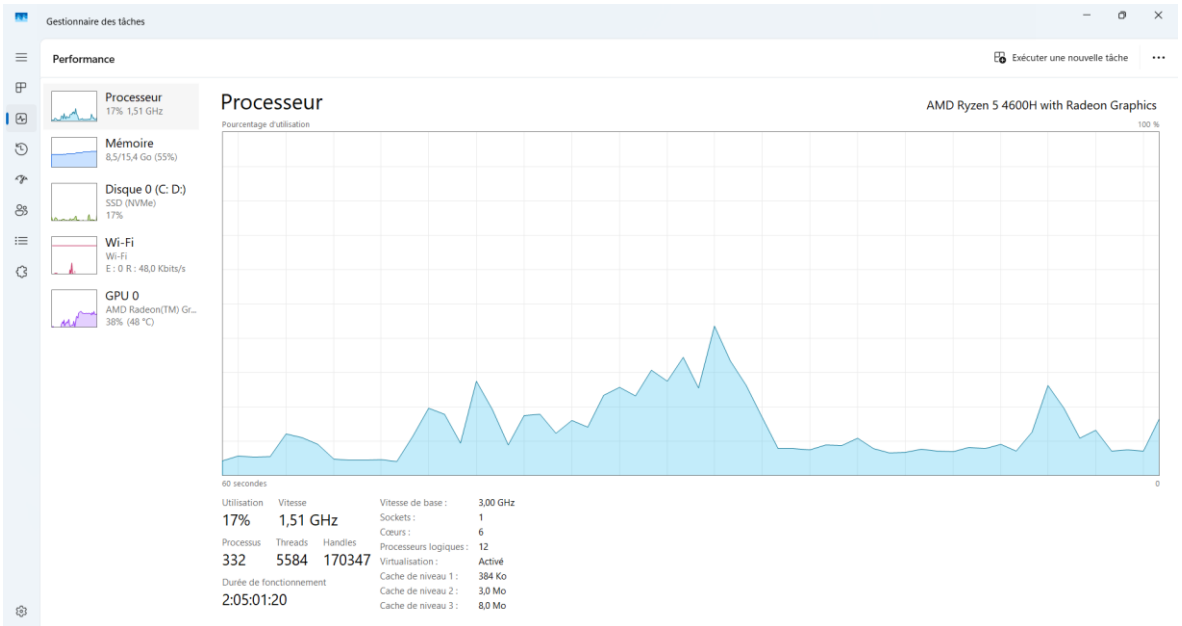
a view where we can see clearly the headlights of the car “on”



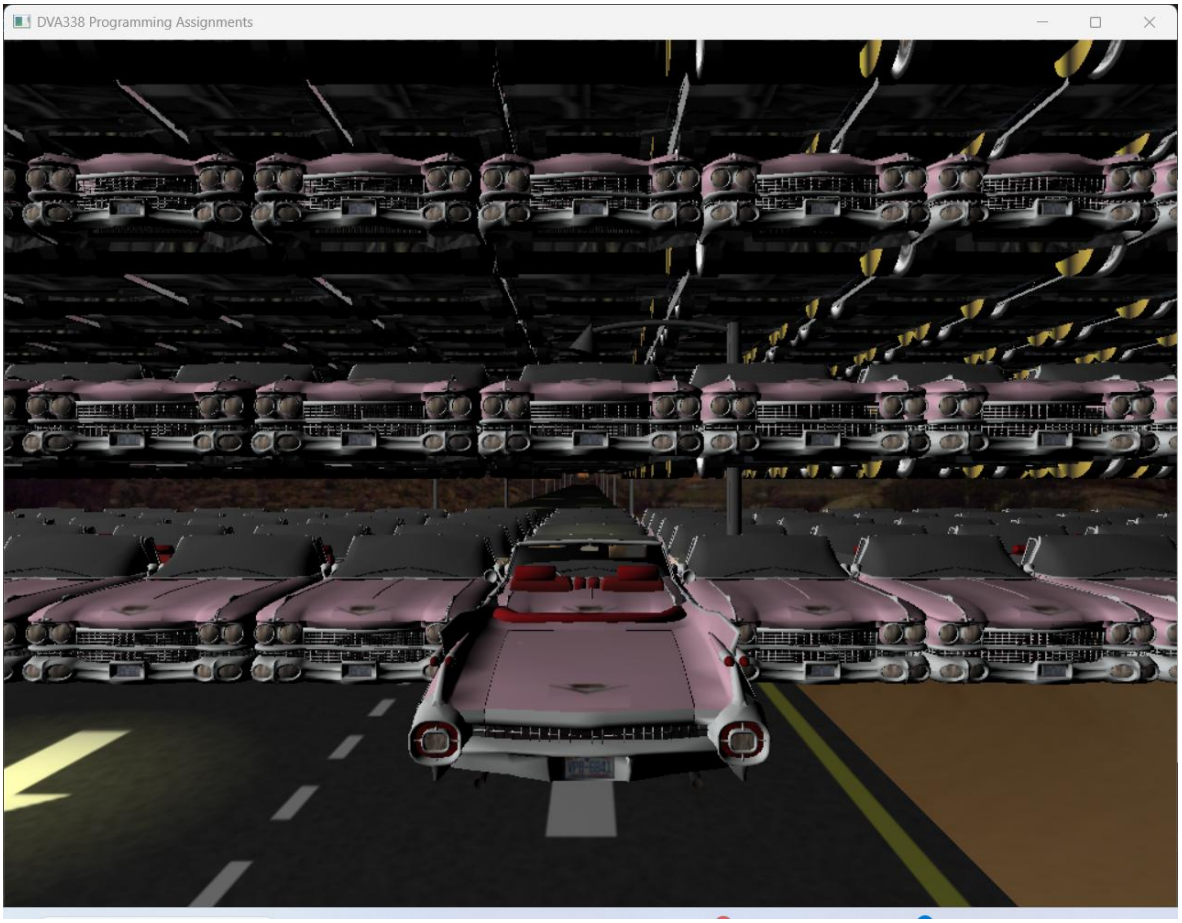
a view where we can see clearly the headlights of the car “off”



a screenshot where we can see the processor, and on the left smaller, we can see GPU 0 before executing the program,



a screenshot where we can see the processor (which is on 17% so not too bad) , and on the left side, we can see GPU 0 after executing the program (which is at 38% so still okay).



we are also able to show 2000 cadillac without crashing, although the frame rate is very bad, since we are still trying to move



the program is still able to switch between shaders, like shown in the

image above, which uses the interlaced shader given in the assignment 2

V) Conclusion and Reflections

Using OpenGL and C++, I successfully made a 3D rendering engine that supports scene changes, lighting, animation and the management of various objects in real time. The addition of obj library allowed the program to import .obj models with all their geometry, materials and textures. It took little effort to put in the Cadillac, road tiles and lamp posts and they were all lit with realistic Phong light. By using obj pointers and my own Instance structure, I could have many separate animations of the same object without using extra space in memory. Its most important feature is that it runs animation using scrolling. The mix of first- and third-person angles and the options to use or change shaders and spotlights added fun interaction, along with proving good control over the rendering in real time.

By working on this project, I reminded myself of many main ideas in developing real-time graphics. I realized that the sequence of applying rotations, translations and scaling is very important, since doing them wrong often causes the graphics to look jerkier or broken. It was also vital to connect vertex attributes and material uniforms properly, because **only missing one connection** could cause problems. The handling of color and texture using modular shader logic showed that the design of the pipeline had to be flexible from the very beginning.

It was especially hard to create and fine-tune the lighting system to work properly in different scenes. To use both point lights and spotlights, I needed to plan my GLSL code and keep uniform names for them. Spotlights, especially car headlights, needed focus angles, an easy shift from the inner to outer cones and smooth switching of their runtime using commands from the keyboard. It was not easy to make sure the scene, objects in motion and camera all matched in lighting and **movement**. The experience provided insights into GPU and GPU optimization, animation with matrix, all of which are essential in the field of real-time graphics development.

References

this book helped me through the assignment 3 : OpenGL Programming Guide, The Official Guide to Learning OpenGL, Version 4.5, written by John Kessenich, Graham Sellers, Dave Shreiner, as well as the youtube video Making Spotlights Work in OpenGL (GLSL) wich explained spotlight cone calculations and shader struct usage for multiple dynamic lights.